

K-Pop Universe - Product Requirements Document (PRD)

Version: 3.0 **Last Updated:** October 31, 2025 **Project Type:** Reddit-Style K-Pop Community Platform **Tech Stack:** Ruby on Rails 7.x, PostgreSQL, Stimulus.js, Hotwire (Turbo), AWS

Table of Contents

- Executive Summary
- User Stories & Use Cases
- Data Models & Database Schema
- API Endpoints & Routes
- Authentication & Authorization
- Frontend Architecture
- Responsive Design Strategy
- Cloud Infrastructure
- Key Features Breakdown
- Implementation Phases

Executive Summary

Project Vision

K-Pop Universe is a mobile-first, Reddit-style community platform dedicated to K-Pop fans worldwide. The platform combines community-driven discussion (similar to Reddit) with Instagram-style engagement mechanics and a comprehensive K-Pop group discovery system.

Core Value Propositions

- Community-First:** Fan-created communities for every K-Pop topic imaginable
- Discovery:** Comprehensive database of K-Pop groups with rich profiles
- Mobile-Optimized:** Built mobile-first with exceptional touch-friendly UX
- Real-Time Engagement:** Pre-built feeds with real-time updates
- Accessible:** Free browsing for guests, account required for participation

Key Success Metrics

- User Registration Conversion Rate: >15% from guest to member
- Daily Active Users (DAU): Target 10K+ in first 6 months
- Average Session Duration: >8 minutes
- Post Engagement Rate: >30% of posts receive hearts/comments
- Mobile Traffic: >70% of total traffic

Technical Architecture Highlights

- Backend:** Ruby on Rails 7.x with PostgreSQL
- Real-Time:** ActionCable + Hotwire Turbo Streams
- Feed System:** Denormalized, pre-built feeds with push-based updates
- Frontend:** Stimulus.js controllers with minimal JavaScript
- Hosting:** AWS (EC2, RDS, S3, CloudFront)
- Future:** Hotwire Native for iOS/Android (Phase 2)

User Stories & Use Cases

User Personas

1. Guest Browser (Unauthenticated)

Demographics: New visitors, lurkers, international fans **Goals:** Discover K-Pop content, explore communities without commitment **Behaviors:**

- Browses homepage trending posts
- Explores "Get to Know Groups" discovery section
- Views posts and comments (read-only)
- Filters groups by company/type

Key User Stories:

- As a guest, I can browse all community posts without creating an account
- As a guest, I can search and filter K-Pop groups by entertainment company
- As a guest, I see prominent CTAs encouraging me to sign up after 3+ interactions
- As a guest, I can view detailed group profiles including members and discography

Mobile Considerations: Fast page loads (<2s), minimal modals/interruptions, sticky signup CTA at bottom

2. Community Member (Registered User)

Demographics: Active K-Pop fans, ages 16-35, global audience **Goals:** Engage with community, share content, discover new groups **Behaviors:**

- Creates posts (text, images, videos, links, polls)
- Hearts and comments on posts
- Shares posts to other communities or externally
- Follows/joins communities
- Builds karma through engagement
- Customizes profile with favorite groups and flair

Key User Stories:

- As a member, I can create a post in any community I've joined with rich media (images/videos)
- As a member, I can heart posts and comments to show appreciation
- As a member, I can share posts to other communities or social media
- As a member, I receive real-time notifications when someone replies to my comments
- As a member, I can customize my home feed by following specific communities
- As a member, I can filter my feed by "Hot", "New", "Top", or "Following"
- As a member, I earn karma points from hearts and engagement on my content
- As a member, I can save/bookmark posts to read later

Mobile Considerations:

- Floating Action Button (FAB) for quick post creation
 - Swipe gestures for navigation between feeds
 - Bottom sheet for quick actions (share, save, report)
 - Touch-friendly voting buttons (min 44x44px)
-

3. Community Moderator

Demographics: Trusted community members, volunteer role **Goals:** Maintain community quality, enforce rules, handle reports **Behaviors:**

- Reviews reported posts/comments
- Removes rule-breaking content
- Pins important announcements
- Sets community guidelines
- Bans/warns problematic users

Key User Stories:

- As a moderator, I can remove posts and comments that violate community rules
- As a moderator, I can pin important posts to the top of the community
- As a moderator, I can view a moderation queue of reported content
- As a moderator, I can ban users from the community (not site-wide)
- As a moderator, I can edit community rules and description
- As a moderator, I receive notifications for new reports

Mobile Considerations: Streamlined moderation UI with swipe-to-action, priority notifications

4. Community Creator/Admin

Demographics: Passionate fans who want to start new communities **Goals:** Build and grow niche K-Pop communities **Behaviors:**

- Creates new communities instantly (no approval needed)
- Customizes community appearance and settings
- Appoints moderators
- Monitors community growth analytics

Key User Stories:

- As a creator, I can instantly create a new community with a unique name/URL
- As a creator, I can upload a community icon and banner image
- As a creator, I can set community visibility (public/private)
- As a creator, I can appoint other users as moderators
- As a creator, I can view community growth metrics (members, posts, engagement)
- As a creator, I can transfer ownership to another user

Mobile Considerations: Community creation wizard optimized for mobile, image cropping tools

Core User Journeys

Journey 1: Discovery & Onboarding Flow

Guest User
↓
Lands on Homepage (trending posts from all communities)
↓
Browses "Get to Know Groups" → Filters by Company (HYBE, SM, YG, JYP)
↓
Clicks on BTS group profile → Views detailed info, photos, links
↓
Sees interesting discussion post about comeback
↓
Clicks "Heart" → Redirected to Sign Up page
↓
Signs up via Google/Apple/Email
↓
Onboarding: "Follow 3 communities to personalize your feed"
↓
Presented with personalized feed of followed communities

Key Touchpoints:

- Homepage must load <2s on 3G connections
- "Get to Know Groups" must be prominent in navigation
- Sign up flow <30 seconds with social login
- Onboarding completable in <60 seconds

Journey 2: Content Creation (Mobile-Optimized)

Logged-In User browsing feed
↓
Taps Floating Action Button (FAB) in bottom-right
↓
Selects Community from list (recently joined shown first)
↓
Chooses Post Type: Text / Image / Video / Link / Poll
↓
Writes post with mobile-optimized editor (supports emoji, markdown)
↓
Uploads images (max 10) or video (max 60s)
↓
Adds optional flair/tags (e.g., "Discussion", "News", "Fan Art")
↓
Taps "Post" → Success animation
↓
Redirected to post detail page → Sees first hearts coming in
↓
Receives push notification: "Your post got 10 hearts!"

Key Touchpoints:

- FAB must be accessible with thumb on all screen sizes
- Image upload supports multi-select from camera roll
- Video upload with progress indicator
- Draft auto-save every 5 seconds
- Post preview before publishing

Journey 3: Community Engagement Loop



Key Touchpoints:

- Feed must load <1s (pre-built architecture)
- Voting/hearting must feel instant (optimistic UI)
- Real-time comment updates via ActionCable
- Infinite scroll with skeleton loaders
- Pull-to-refresh with visual feedback

Mobile vs Desktop Experience Comparison

Feature	Mobile Experience	Desktop Experience
Navigation	Bottom tab bar (Home, Discover, Notifications, Profile) or hamburger menu	Persistent left sidebar with community list
Post Creation	Floating Action Button (FAB)	Prominent "Create Post" button in header
Voting/Hearts	Large touch targets (48x48px), swipe gestures	Standard buttons (hover effects)
Content Display	Single column, card-based (16:9 images)	Multi-column layout (feed + sidebar)
Comments	Full-screen modal or bottom sheet	Inline expansion or side panel
Search	Full-screen search overlay	Header search with dropdown
Moderation	Core actions only (remove, pin, ban)	Full moderation dashboard
Group Discovery	Swipeable cards, bottom sheet filters	Grid layout with sidebar filters
Notifications	Push notifications + in-app notification center	Browser notifications + dropdown
Profile	Full-screen profile view with tabs	Two-column layout (info + activity)

Engagement & Retention Mechanics

Post-Signup Engagement Hooks

1. Onboarding Flow:
- "Follow 3 communities to get started" (required step)
 - "Add your favorite K-Pop groups to your profile" (optional)
 - "Enable notifications to never miss updates" (permission prompt)
2. Gamification - Karma System:
- Earn karma from hearts on posts/comments
 - Karma thresholds unlock badges (Bronze: 100, Silver: 500, Gold: 1000)
 - Display karma on profile and next to username

3. **Notifications** (Critical for retention):
 - Comment replies (instant push)
 - Post hearts (batched: "Your post got 10 hearts!")
 - Milestone notifications ("Your post reached 100 hearts!")
 - Community updates (weekly digest: "5 new posts in r/BTS")
4. **Profile Building**:
 - User flair (text next to username, e.g., "ARMY since 2013")
 - Favorite groups displayed on profile
 - Achievement badges (e.g., "Early Adopter", "Top Contributor")
 - Streak tracking (days active)
5. **Content Creation Incentives**:
 - "Your first post gets you 10 bonus karma!"
 - Easy cross-posting to other communities
 - Post templates for common types (e.g., "Comeback Discussion")

Guest → Member Conversion Triggers

1. **Persistent CTA Bar**:
 - Bottom sticky bar (mobile): "Sign up to customize your feed"
 - Top banner (desktop): "Join 50K+ K-Pop fans on K-Pop Universe"
2. **Feature Gating**:
 - After 3 post views: "Create account to save posts"
 - After 5 minutes: "Sign up to follow your favorite communities"
 - After clicking heart: Immediate signup modal
3. **Social Proof**:
 - "Join 50,000+ K-Pop fans"
 - "12,345 posts created today"
 - "Trending: 500 people discussing [Group Name]"
4. **Time-Based**:
 - After 3rd visit: "Welcome back! Sign up to pick up where you left off"
 - After 7 days: "You've been here 5 times! Join us?"

Data Models & Database Schema

Entity Relationship Overview



Core Models

1. User Model

```

# app/models/user.rb
class User < ApplicationRecord
  # Devise modules
  devise :database_authenticatable, :registerable, :recoverable,
         :rememberable, :validatable, :trackable, :omniauthable,
         omniauth_providers: [:google_oauth2, :apple, :facebook]

  # Associations
  has_many :posts, dependent: :destroy
  has_many :comments, dependent: :destroy
  has_many :hearts, dependent: :destroy
  has_many :community_memberships, dependent: :destroy
  has_many :communities, through: :community_memberships
  has_many :owned_communities, class_name: 'Community', foreign_key: 'creator_id'
  has_many :moderator_roles, dependent: :destroy
  has_many :moderated_communities, through: :moderator_roles, source: :community
  has_many :notifications, dependent: :destroy
  has_many :feed_items, dependent: :destroy
  has_many :followed_groups, dependent: :destroy
  has_many :kpop_groups, through: :followed_groups
  has_many :saved_posts, dependent: :destroy

  # Validations
  validates :username, presence: true, uniqueness: true,
                    format: { with: /\A[a-zA-Z0-9_]{3,20}\z/ }
  validates :display_name, length: { maximum: 50 }
  validates :bio, length: { maximum: 500 }
end

```

Database Schema:

```

CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  encrypted_password VARCHAR(255) NOT NULL,
  username VARCHAR(20) NOT NULL UNIQUE,
  display_name VARCHAR(50),
  bio TEXT,
  avatar_url TEXT,
  karma_points INTEGER DEFAULT 0,
  role VARCHAR(20) DEFAULT 'member', -- 'member', 'admin', 'banned'

  -- Devise fields
  reset_password_token VARCHAR(255),
  reset_password_sent_at TIMESTAMP,
  remember_created_at TIMESTAMP,
  sign_in_count INTEGER DEFAULT 0,
  current_sign_in_at TIMESTAMP,
  last_sign_in_at TIMESTAMP,
  current_sign_in_ip INET,
  last_sign_in_ip INET,
  confirmation_token VARCHAR(255),
  confirmed_at TIMESTAMP,
  confirmation_sent_at TIMESTAMP,

  -- OAuth fields
  provider VARCHAR(50),
  uid VARCHAR(255),

  -- Preferences
  email_notifications BOOLEAN DEFAULT true,
  push_notifications BOOLEAN DEFAULT true,

  -- Onboarding
  onboarding_completed BOOLEAN DEFAULT false,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_users_username (username),
  INDEX idx_users_email (email),
  INDEX idx_users_karma (karma_points DESC)
);

```

2. Community Model

```
# app/models/community.rb
class Community < ApplicationRecord
  belongs_to :creator, class_name: 'User'

  has_many :community_memberships, dependent: :destroy
  has_many :members, through: :community_memberships, source: :user
  has_many :posts, dependent: :destroy
  has_many :moderator_roles, dependent: :destroy
  has_many :moderators, through: :moderator_roles, source: :user

  validates :name, presence: true, uniqueness: true,
    format: { with: /\A[a-zA-Z0-9]{3,21}\z/ }
  validates :display_name, presence: true, length: { maximum: 100 }
  validates :description, length: { maximum: 500 }
  validates :rules, length: { maximum: 5000 }

  enum visibility: { public_community: 0, private_community: 1 }
  enum :content_type, { all_content: 0, text_only: 1, media_only: 2 }
end
```

Database Schema:

```
CREATE TABLE communities (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(21) NOT NULL UNIQUE, -- URL slug (e.g., "bts")
  display_name VARCHAR(100) NOT NULL, -- Display title (e.g., "BTS - Bangtan Sonyeondan")
  description TEXT,
  rules TEXT,
  icon_url TEXT,
  banner_url TEXT,

  creator_id BIGINT NOT NULL REFERENCES users(id),

  -- Settings
  visibility INTEGER DEFAULT 0, -- 0: public, 1: private
  content_type INTEGER DEFAULT 0, -- 0: all, 1: text_only, 2: media_only
  allow_polls BOOLEAN DEFAULT true,
  require_flair BOOLEAN DEFAULT false,

  -- Stats (cached for performance)
  members_count INTEGER DEFAULT 0,
  posts_count INTEGER DEFAULT 0,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_communities_name (name),
  INDEX idx_communities_creator (creator_id),
  INDEX idx_communities_members_count (members_count DESC)
);
```

3. Post Model

```
# app/models/post.rb
class Post < ApplicationRecord
  belongs_to :user
  belongs_to :community

  has_many :comments, dependent: :destroy
  has_many :hearts, as: :heartable, dependent: :destroy
  has_many :post_images, dependent: :destroy
  has_many :post_videos, dependent: :destroy
  has_many :shares, dependent: :destroy
  has_many :saved_posts, dependent: :destroy

  accepts_nested_attributes_for :post_images, :post_videos

  validates :title, presence: true, length: { maximum: 300 }
  validates :body, length: { maximum: 40000 }
  validates :post_type, presence: true

  enum post_type: { text: 0, link: 1, image: 2, video: 3, poll: 4 }

  # Cached counters for performance
  def hearts_count
    self[:hearts_count] || hearts.count
  end

  def comments_count
    self[:comments_count] || comments.count
  end
end
```

Database Schema:

```

CREATE TABLE posts (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id),
  community_id BIGINT NOT NULL REFERENCES communities(id),

  title VARCHAR(300) NOT NULL,
  body TEXT,
  post_type INTEGER NOT NULL, -- 0: text, 1: link, 2: image, 3: video, 4: poll

  -- Optional fields based on post_type
  link_url TEXT,
  flair VARCHAR(50),

  -- Cached engagement metrics (updated via background jobs)
  hearts_count INTEGER DEFAULT 0,
  comments_count INTEGER DEFAULT 0,
  shares_count INTEGER DEFAULT 0,
  engagement_score FLOAT DEFAULT 0.0, -- Calculated for feed ranking

  -- Moderation
  is_pinned BOOLEAN DEFAULT false,
  is_removed BOOLEAN DEFAULT false,
  removed_reason TEXT,

  -- Flags
  is_nsfw BOOLEAN DEFAULT false,
  is_spoiler BOOLEAN DEFAULT false,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_posts_user (user_id),
  INDEX idx_posts_community (community_id),
  INDEX idx_posts_created_at (created_at DESC),
  INDEX idx_posts_engagement (engagement_score DESC),
  INDEX idx_posts_community_created (community_id, created_at DESC)
);

```

4. Comment Model

```

# app/models/comment.rb
class Comment < ApplicationRecord
  belongs_to :user
  belongs_to :post
  belongs_to :parent, class_name: 'Comment', optional: true

  has_many :replies, class_name: 'Comment', foreign_key: 'parent_id', dependent: :destroy
  has_many :hearts, as: :heartable, dependent: :destroy

  validates :body, presence: true, length: { maximum: 10000 }

  # Nested comment depth tracking
  before_create :set_depth

  private

  def set_depth
    self.depth = parent ? parent.depth + 1 : 0
  end
end

```

Database Schema:

```

CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id),
  post_id BIGINT NOT NULL REFERENCES posts(id),
  parent_id BIGINT REFERENCES comments(id), -- NULL for top-level comments

  body TEXT NOT NULL,
  depth INTEGER DEFAULT 0, -- Nesting level (0 = top-level)

  -- Cached metrics
  hearts_count INTEGER DEFAULT 0,

  -- Moderation
  is_removed BOOLEAN DEFAULT false,
  removed_reason TEXT,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_comments_post (post_id, created_at DESC),
  INDEX idx_comments_user (user_id),
  INDEX idx_comments_parent (parent_id)
);

```

5. Heart Model (Polymorphic for Posts & Comments)

```

# app/models/heart.rb
class Heart < ApplicationRecord
  belongs_to :user
  belongs_to :heartable, polymorphic: true

  validates :user_id, uniqueness: { scope: [:heartable_type, :heartable_id] }

  # After create, update cached counter and karma
  after_create :increment_counters
  after_destroy :decrement_counters

  private

  def increment_counters
    heartable.increment!(:hearts_count)
    heartable.user.increment!(:karma_points)
  end

  def decrement_counters
    heartable.decrement!(:hearts_count)
    heartable.user.decrement!(:karma_points)
  end
end

```

Database Schema:

```
CREATE TABLE hearts (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id),
  heartable_type VARCHAR(50) NOT NULL, -- 'Post' or 'Comment'
  heartable_id BIGINT NOT NULL,

  created_at TIMESTAMP NOT NULL,

  UNIQUE INDEX idx_hearts_unique (user_id, heartable_type, heartable_id),
  INDEX idx_hearts_heartable (heartable_type, heartable_id)
);
```

6. FeedItem Model (Denormalized for Performance)

```
# app/models/feed_item.rb
class FeedItem < ApplicationRecord
  belongs_to :user
  belongs_to :post

  validates :user_id, presence: true
  validates :post_id, presence: true

  # Denormalized fields from post (cached at creation)
  # Updated periodically via background jobs
end
```

Database Schema:

```
CREATE TABLE feed_items (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id),
  post_id BIGINT NOT NULL REFERENCES posts(id),

  -- Denormalized post data (updated at thresholds)
  post_title VARCHAR(300),
  post_type INTEGER,
  community_id BIGINT,
  community_name VARCHAR(21),
  author_id BIGINT,
  author_username VARCHAR(20),

  -- Cached metrics (synced at thresholds: 10, 50, 100, 500, 1000, 5000)
  cached_hearts_count INTEGER DEFAULT 0,
  cached_comments_count INTEGER DEFAULT 0,

  -- Feed positioning
  priority_score FLOAT DEFAULT 0.0, -- Calculated based on engagement + recency
  position INTEGER, -- For stable pagination

  -- Tracking
  is_seen BOOLEAN DEFAULT false,
  seen_at TIMESTAMP,

  created_at TIMESTAMP NOT NULL, -- When added to user's feed

  INDEX idx_feed_user_priority (user_id, priority_score DESC),
  INDEX idx_feed_user_created (user_id, created_at DESC),
  INDEX idx_feed_post (post_id),
  UNIQUE INDEX idx_feed_user_post (user_id, post_id)
);
```

Feed Update Strategy:

- When user follows community → Backfill top 50 recent posts into their feed
- When new post is created → Push to feeds of all community members (background job)
- When post crosses engagement thresholds (10, 50, 100, 500, 1000 hearts) → Update cached metrics across all feed_items
- Priority score recalculated: $\log(\text{hearts} + 1) + \log(\text{comments} + 1) - (\text{hours_since_creation} / 24)$

7. KpopGroup Model (Discovery Section)

```
# app/models/kpop_group.rb
class KpopGroup < ApplicationRecord
  belongs_to :company

  has_many :group_members, dependent: :destroy
  has_many :followed_groups, dependent: :destroy
  has_many :followers, through: :followed_groups, source: :user

  validates :name, presence: true, uniqueness: true
  validates :debut_date, presence: true

  enum group_type: { boy_group: 0, girl_group: 1, coed_group: 2, soloist: 3 }
  enum status: { active: 0, disbanded: 1, hiatus: 2 }
end
```

Database Schema:

```
CREATE TABLE kpop_groups (
  id BIGSERIAL PRIMARY KEY,
  company_id BIGINT REFERENCES companies(id),

  name VARCHAR(100) NOT NULL UNIQUE,
  korean_name VARCHAR(100),
  debut_date DATE NOT NULL,
  group_type INTEGER NOT NULL, -- 0: boy_group, 1: girl_group, 2: coed, 3: soloist
  status INTEGER DEFAULT 0, -- 0: active, 1: disbanded, 2: hiatus

  -- Profile details
  description TEXT,
  fandom_name VARCHAR(50),
  official_colors VARCHAR(50), -- e.g., "#7B68EE, #FFB6C1"

  -- Media
  profile_image_url TEXT,
  banner_image_url TEXT,

  -- Links
  youtube_url TEXT,
  twitter_url TEXT,
  instagram_url TEXT,
  tiktok_url TEXT,
  spotify_url TEXT,
  official_website TEXT,

  -- Stats
  followers_count INTEGER DEFAULT 0,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_kpop_groups_company (company_id),
  INDEX idx_kpop_groups_type (group_type),
  INDEX idx_kpop_groups_debut (debut_date DESC),
  INDEX idx_kpop_groups_name (name)
);
```

8. Company Model

```
# app/models/company.rb
class Company < ApplicationRecord
  has_many :kpop_groups, dependent: :nullify

  validates :name, presence: true, uniqueness: true
end
```

Database Schema:

```
CREATE TABLE companies (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  korean_name VARCHAR(100),
  founded_year INTEGER,
  description TEXT,
  logo_url TEXT,
  website TEXT,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_companies_name (name)
);
```

9. Additional Supporting Models

GroupMember:

```
CREATE TABLE group_members (
  id BIGSERIAL PRIMARY KEY,
  kpop_group_id BIGINT NOT NULL REFERENCES kpop_groups(id),

  stage_name VARCHAR(50) NOT NULL,
  full_name VARCHAR(100),
  birth_date DATE,
  position VARCHAR(100), -- e.g., "Main Vocalist, Visual"
  nationality VARCHAR(50),
  profile_image_url TEXT,

  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL,

  INDEX idx_group_members_group (kpop_group_id)
);
```

PostImage:

```
CREATE TABLE post_images (
  id BIGSERIAL PRIMARY KEY,
  post_id BIGINT NOT NULL REFERENCES posts(id),
  image_url TEXT NOT NULL,
  thumbnail_url TEXT,
  display_order INTEGER DEFAULT 0,
  width INTEGER,
  height INTEGER,

  created_at TIMESTAMP NOT NULL,

  INDEX idx_post_images_post (post_id)
);
```

PostVideo:

```
CREATE TABLE post_videos (  
  id BIGSERIAL PRIMARY KEY,  
  post_id BIGINT NOT NULL REFERENCES posts(id),  
  video_url TEXT NOT NULL,  
  thumbnail_url TEXT,  
  duration_seconds INTEGER,  
  is_external BOOLEAN DEFAULT false, -- True for YouTube/TikTok embeds  
  
  created_at TIMESTAMP NOT NULL,  
  
  INDEX idx_post_videos_post (post_id)  
);
```

Notification:

```
CREATE TABLE notifications (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL REFERENCES users(id),  
  
  notification_type VARCHAR(50) NOT NULL, -- 'comment_reply', 'post_heart', 'mention', 'milestone'  
  
  -- Polymorphic reference to the subject  
  subject_type VARCHAR(50), -- 'Post', 'Comment', etc.  
  subject_id BIGINT,  
  
  -- Actor (who triggered the notification)  
  actor_id BIGINT REFERENCES users(id),  
  
  -- Content  
  message TEXT,  
  
  -- Status  
  is_read BOOLEAN DEFAULT false,  
  read_at TIMESTAMP,  
  
  created_at TIMESTAMP NOT NULL,  
  
  INDEX idx_notifications_user_unread (user_id, is_read, created_at DESC),  
  INDEX idx_notifications_user_created (user_id, created_at DESC)  
);
```

CommunityMembership:

```
CREATE TABLE community_memberships (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL REFERENCES users(id),  
  community_id BIGINT NOT NULL REFERENCES communities(id),  
  
  joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  UNIQUE INDEX idx_membership_unique (user_id, community_id),  
  INDEX idx_membership_community (community_id, joined_at DESC)  
);
```

ModeratorRole:

```
CREATE TABLE moderator_roles (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL REFERENCES users(id),  
  community_id BIGINT NOT NULL REFERENCES communities(id),  
  
  appointed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  appointed_by_id BIGINT REFERENCES users(id),  
  
  UNIQUE INDEX idx_moderator_unique (user_id, community_id),  
  INDEX idx_moderator_community (community_id)  
);
```

API Endpoints & Routes

RESTful Routes

Authentication (Devise)

```
# config/routes.rb  
devise_for :users, controllers: {  
  registrations: 'users/registrations',  
  sessions: 'users/sessions',  
  omniauth_callbacks: 'users/omniauth_callbacks'  
}  
  
# Custom auth endpoints  
namespace :auth do  
  post 'social_login/:provider', to: 'social#create' # Google, Apple, Facebook  
  delete 'logout', to: 'sessions#destroy'  
  get 'verify_token', to: 'sessions#verify' # For mobile apps  
end
```

Endpoints:

- `POST /users` - Sign up with email/password
- `POST /users/sign_in` - Log in
- `DELETE /users/sign_out` - Log out
- `POST /auth/social_login/google_oauth2` - Social login
- `POST /auth/social_login/apple`
- `POST /auth/social_login/facebook`

Users

```
resources :users, only: [:show, :update] do  
  member do  
    get 'posts' # User's post history  
    get 'comments' # User's comment history  
    get 'saved' # Saved posts  
    get 'followers'  
    post 'follow'  
    delete 'unfollow'  
  end  
end
```

Endpoints:

- `GET /users/:id` - User profile
 - `PATCH /users/:id` - Update profile
 - `GET /users/:id/posts` - User's posts
 - `GET /users/:id/comments` - User's comments
 - `GET /users/:id/saved` - Saved posts
-

Communities

```
resources :communities do
  member do
    post 'join'
    delete 'leave'
    get 'moderators'
    post 'moderators/:user_id', to: 'communities#add_moderator', as: 'add_moderator'
  end

  resources :posts, only: [:index, :create]
end
```

Endpoints:

- GET /communities - List all communities (with filters)
 - POST /communities - Create community
 - GET /communities/:id - Community detail
 - PATCH /communities/:id - Update community (moderators only)
 - DELETE /communities/:id - Delete community (creator only)
 - POST /communities/:id/join - Join community
 - DELETE /communities/:id/leave - Leave community
 - GET /communities/:id/posts - Posts in community
 - POST /communities/:id/posts - Create post in community
-

Posts

```
resources :posts do
  member do
    post 'heart'
    delete 'unheart'
    post 'share'
    post 'save'
    delete 'unsave'
    patch 'pin' # Moderators only
    patch 'remove' # Moderators only
  end

  resources :comments, only: [:index, :create]
end
```

Endpoints:

- GET /posts/:id - Post detail
 - PATCH /posts/:id - Update post (author only)
 - DELETE /posts/:id - Delete post (author or moderator)
 - POST /posts/:id/heart - Heart post
 - DELETE /posts/:id/unheart - Unheart post
 - POST /posts/:id/share - Share post
 - POST /posts/:id/save - Save post
 - GET /posts/:id/comments - Comments on post
 - POST /posts/:id/comments - Add comment
-

Comments

```
resources :comments, only: [:update, :destroy] do
  member do
    post 'heart'
    delete 'unheart'
  end

  resources :comments, only: [:create] # Nested replies
end
```

Endpoints:

- PATCH /comments/:id - Update comment
- DELETE /comments/:id - Delete comment
- POST /comments/:id/heart - Heart comment
- POST /comments/:id/comments - Reply to comment

Feed

```
namespace :feed do
  get 'home', to: 'feed#home' # Personalized feed
  get 'all', to: 'feed#all' # All communities
  get 'popular', to: 'feed#popular' # Hot/trending
end
```

Endpoints:

- GET /feed/home - Personalized feed (followed communities)
- GET /feed/all - All posts from all communities
- GET /feed/popular - Trending posts (algorithm-based)

Query Parameters:

- filter: hot, new, top (default: hot)
- time: today, week, month, year, all (for top filter)
- page: Pagination (default: 1)
- per_page: Items per page (default: 25, max: 100)

K-Pop Discovery

```
resources :kpop_groups, only: [:index, :show] do
  member do
    post 'follow'
    delete 'unfollow'
  end

  collection do
    get 'search' # Search by name
    get 'filter' # Advanced filters
  end
end

resources :companies, only: [:index, :show]
```

Endpoints:

- GET /kpop_groups - List groups (with filters)
- GET /kpop_groups/:id - Group detail
- POST /kpop_groups/:id/follow - Follow group
- GET /kpop_groups/search?q=BTS - Search groups
- GET /kpop_groups/filter?company_id=1&group_type=boy_group
- GET /companies - List companies
- GET /companies/:id - Company detail with groups

Notifications

```
resources :notifications, only: [:index] do
  member do
    patch 'mark_read'
  end

  collection do
    patch 'mark_all_read'
  end
end
```

Endpoints:

- GET /notifications - List notifications
 - PATCH /notifications/:id/mark_read - Mark as read
 - PATCH /notifications/mark_all_read - Mark all as read
-

Search

```
namespace :search do
  get 'global' # Search across posts, communities, users, groups
  get 'posts'
  get 'communities'
  get 'users'
  get 'kpop_groups'
end
```

Endpoints:

- GET /search/global?q=BTS - Search everything
 - GET /search/posts?q=comeback - Search posts
 - GET /search/communities?q=bts - Search communities
 - GET /search/users?q=john - Search users
 - GET /search/kpop_groups?q=blackpink - Search K-Pop groups
-

Hotwire Turbo Stream Endpoints

For real-time updates without full page reloads:

```
# Real-time post updates
turbo_stream_from "post_#{post.id}" # Hearts, comments

# Real-time feed updates
turbo_stream_from "user_feed_#{current_user.id}" # New posts in feed

# Real-time notification updates
turbo_stream_from "user_notifications_#{current_user.id}"

# Real-time comment thread updates
turbo_stream_from "post_#{post.id}_comments"
```

Turbo Stream Actions:

- append - Add new comments to thread
 - update - Update heart count on post
 - prepend - Add new post to top of feed
 - replace - Replace notification badge count
-

API Response Format (JSON)

Success Response

```
{
  "status": "success",
  "data": {
    "post": {
      "id": 123,
      "title": "BTS Comeback Discussion",
      "body": "...",
      "hearts_count": 42,
      "comments_count": 15,
      "created_at": "2025-10-30T12:00:00Z",
      "user": {
        "id": 1,
        "username": "army_fan",
        "avatar_url": "..."
      },
      "community": {
        "id": 5,
        "name": "bts",
        "display_name": "BTS - Bangtan Sonyeondan"
      }
    }
  },
  "meta": {
    "page": 1,
    "per_page": 25,
    "total_pages": 10,
    "total_count": 250
  }
}
```

Error Response

```
{
  "status": "error",
  "errors": [
    {
      "field": "title",
      "message": "Title can't be blank"
    }
  ],
  "message": "Validation failed"
}
```

Authentication & Authorization

Authentication Strategy (Devise + OmniAuth)

Devise Configuration

```
# config/initializers/devise.rb
Devise.setup do |config|
  config.mailer_sender = 'noreply@kpopuniverse.com'
  config.authentication_keys = [:email]
  config.case_insensitive_keys = [:email]
  config.strip_whitespace_keys = [:email]
  config.skip_session_storage = [:http_auth]
  config.stretches = Rails.env.test? ? 1 : 12
  config.reconfirmable = true
  config.expire_all_remember_me_on_sign_out = true
  config.password_length = 8..128
  config.email_regexp = /\A[^\s]+@[^\s]+\z/
  config.reset_password_within = 6.hours
  config.sign_out_via = :delete
end
```

Social Login (OmniAuth)

```
# config/initializers/omniauth.rb
Rails.application.config.middleware.use OmniAuth::Builder do
  provider :google_oauth2,
    ENV['GOOGLE_CLIENT_ID'],
    ENV['GOOGLE_CLIENT_SECRET'],
    {
      scope: 'email,profile',
      prompt: 'select_account',
      image_aspect_ratio: 'square',
      image_size: 200
    }

  provider :apple,
    ENV['APPLE_CLIENT_ID'],
    '',
    {
      scope: 'email name',
      team_id: ENV['APPLE_TEAM_ID'],
      key_id: ENV['APPLE_KEY_ID'],
      pem: ENV['APPLE_PRIVATE_KEY']
    }

  provider :facebook,
    ENV['FACEBOOK_APP_ID'],
    ENV['FACEBOOK_APP_SECRET'],
    {
      scope: 'email,public_profile',
      info_fields: 'email,name,picture'
    }
end
```

Mobile-Friendly Auth Flow

1. **Guest Browsing:** No friction, immediate access to content
2. **Social Login Priority:** Google/Apple buttons above email/password form
3. **One-Tap Sign-In:** Pre-fill email if possible (browser autocomplete)
4. **Remember Me:** Default to 30-day session for mobile users
5. **Password Reset:** Email link with deep link back to app

Authorization Strategy (Pundit)

User Roles

```
# app/models/user.rb
enum role: {
  member: 0,      # Default role
  moderator: 1,   # Assigned per-community via ModeratorRole
  admin: 2,       # Site-wide admin
  banned: 3       # Banned users
}
```

Policy Structure

PostPolicy:

```
# app/policies/post_policy.rb
class PostPolicy < ApplicationPolicy
  def create?
    user.present? && !user.banned?
  end

  def update?
    user == record.user || community_moderator?
  end

  def destroy?
    user == record.user || community_moderator? || user.admin?
  end

  def pin?
    community_moderator? || user.admin?
  end

  def remove?
    community_moderator? || user.admin?
  end

  private

  def community_moderator?
    record.community.moderators.include?(user)
  end
end
```

CommunityPolicy:

```
# app/policies/community_policy.rb
class CommunityPolicy < ApplicationPolicy
  def create?
    user.present? && !user.banned?
  end

  def update?
    user == record.creator || moderator? || user.admin?
  end

  def destroy?
    user == record.creator || user.admin?
  end

  def add_moderator?
    user == record.creator || user.admin?
  end

  private

  def moderator?
    record.moderators.include?(user)
  end
end
```

Permission Matrix

Action	Guest	Member	Moderator (per-community)	Community Creator	Admin
Browse posts	✓	✓	✓	✓	✓
Create post	✗	✓	✓	✓	✓
Edit own post	✗	✓	✓	✓	✓
Delete own post	✗	✓	✓	✓	✓
Delete any post	✗	✗	✓ (in community)	✓ (in community)	✓
Pin post	✗	✗	✓ (in community)	✓ (in community)	✓
Create community	✗	✓	✓	✓	✓
Edit community	✗	✗	✓ (in community)	✓ (in community)	✓
Add moderator	✗	✗	✗	✓ (in community)	✓
Ban user	✗	✗	✗	✗	✓

Mobile Authentication UX

Sign Up Flow

1. User taps "Sign Up" button
↓
2. Modal/Full-screen form appears
↓
3. Social login buttons (Google, Apple, Facebook)
 - Large touch targets (min 48px height)
 - Brand colors and icons↓
4. OR divider
↓
5. Email/Password form
 - Email input (type="email", autocomplete)
 - Password input (show/hide toggle icon)
 - Username input (check availability in real-time)↓
6. "Create Account" button
↓
7. Onboarding flow (follow 3 communities)

Sign In Flow

1. User taps "Sign In" button
↓
2. Modal appears with social login buttons
↓
3. OR email/password form
↓
4. "Remember Me" checkbox (checked by default)
↓
5. "Sign In" button
↓
6. Redirect back to previous page

Session Management

- **Web:** 30-day cookie with remember_me token
- **Mobile:** Store JWT token in secure storage
- **Auto-refresh:** Silent token refresh before expiry
- **Logout:** Clear all tokens and redirect to homepage

Frontend Architecture

Technology Stack

- **Hotwire Turbo:** SPA-like navigation without full page reloads
- **Stimulus.js:** Lightweight JavaScript controllers for interactivity
- **ViewComponent:** Reusable component-based views
- **Tailwind CSS:** Utility-first CSS framework for rapid styling
- **ActionCable:** WebSocket support for real-time updates

Stimulus Controllers

1. Heart Controller

```

// app/javascript/controllers/heart_controller.js
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static values = {
    heartableType: String,
    heartableId: Number,
    hearted: Boolean
  }

  static targets = ["button", "count"]

  connect() {
    this.updateUI()
  }

  async toggle(event) {
    event.preventDefault()

    // Optimistic UI update
    this.heartedValue = !this.heartedValue
    this.updateUI()

    const url = this.heartedValue
      ? `/posts/${this.heartableIdValue}/heart`
      : `/posts/${this.heartableIdValue}/unheart`

    try {
      const response = await fetch(url, {
        method: this.heartedValue ? 'POST' : 'DELETE',
        headers: {
          'X-CSRF-Token': this.csrfToken(),
          'Content-Type': 'application/json'
        }
      })
    })

    if (!response.ok) {
      // Revert on error
      this.heartedValue = !this.heartedValue
      this.updateUI()
    } else {
      const data = await response.json()
      this.countTarget.textContent = data.hearts_count
    }
  } catch (error) {
    // Revert on error
    this.heartedValue = !this.heartedValue
    this.updateUI()
  }
}

updateUI() {
  if (this.heartedValue) {
    this.buttonTarget.classList.add('hearted')
    this.buttonTarget.innerHTML = '♥'
  } else {
    this.buttonTarget.classList.remove('hearted')
    this.buttonTarget.innerHTML = '💔'
  }
}

csrfToken() {
  return document.querySelector('[name="csrf-token"]').content
}
}

```

Usage:

```
<div data-controller="heart"
  data-heart-heartable-type-value="Post"
  data-heart-heartable-id-value="<%= post.id %>"
  data-heart-heartable-id-value="<%= current_user&.hearted?(post) %>">
  <button data-heart-target="button"
    data-action="click->heart#toggle"
    class="heart-button">
    
  </button>
  <span data-heart-target="count"><%= post.hearts_count %></span>
</div>
```

2. Infinite Scroll Controller

```
// app/javascript/controllers/infinite_scroll_controller.js
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static values = {
    url: String,
    page: { type: Number, default: 1 },
    loading: { type: Boolean, default: false }
  }
  static targets = ["entries", "pagination", "loader"]

  connect() {
    this.intersectionObserver = new IntersectionObserver(
      entries => this.handleIntersection(entries),
      { threshold: 0.1 }
    )

    if (this.hasPaginationTarget) {
      this.intersectionObserver.observe(this.paginationTarget)
    }
  }

  disconnect() {
    this.intersectionObserver.disconnect()
  }

  async handleIntersection(entries) {
    entries.forEach(async entry => {
      if (entry.isIntersecting && !this.loadingValue) {
        await this.loadMore()
      }
    })
  }

  async loadMore() {
    if (this.loadingValue) return

    this.loadingValue = true
    this.showLoader()

    this.pageValue += 1
    const url = `${this.urlValue}?page=${this.pageValue}`

    try {
      const response = await fetch(url, {
        headers: { 'Accept': 'text/vnd.turbo-stream.html' }
      })
    }
  }
}
```

```
    if (response.ok) {
      const html = await response.text()
      Turbo.renderStreamMessage(html)
    }
  } catch (error) {
    console.error('Failed to load more:', error)
  } finally {
    this.loadingValue = false
    this.hideLoader()
  }
}

showLoader() {
  if (this.hasLoaderTarget) {
    this.loaderTarget.classList.remove('hidden')
  }
}

hideLoader() {
  if (this.hasLoaderTarget) {
    this.loaderTarget.classList.add('hidden')
  }
}
}
```

3. Dropdown Controller (Mobile-Friendly)

```
// app/javascript/controllers/dropdown_controller.js
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["menu", "button"]

  connect() {
    this.closeOnClickOutside = this.closeOnClickOutside.bind(this)
  }

  toggle(event) {
    event.stopPropagation()

    if (this.menuTarget.classList.contains('hidden')) {
      this.open()
    } else {
      this.close()
    }
  }

  open() {
    this.menuTarget.classList.remove('hidden')
    document.addEventListener('click', this.closeOnClickOutside)
  }

  close() {
    this.menuTarget.classList.add('hidden')
    document.removeEventListener('click', this.closeOnClickOutside)
  }

  closeOnClickOutside(event) {
    if (!this.element.contains(event.target)) {
      this.close()
    }
  }
}
```

4. Image Upload Controller (Mobile Camera Support)

```
// app/javascript/controllers/image_upload_controller.js
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["input", "preview", "progressBar"]
  static values = { maxFiles: { type: Number, default: 10 } }

  connect() {
    this.selectedFiles = []
  }

  selectFiles(event) {
    const files = Array.from(event.target.files)

    if (files.length + this.selectedFiles.length > this.maxFilesValue) {
      alert(`You can only upload up to ${this.maxFilesValue} images`)
      return
    }

    files.forEach(file => {
      if (file.type.startsWith('image/')) {
        this.selectedFiles.push(file)
        this.showPreview(file)
      }
    })
  }
}
```

```

    })
  }

  showPreview(file) {
    const reader = new FileReader()

    reader.onload = (e) => {
      const div = document.createElement('div')
      div.className = 'relative inline-block m-2'
      div.innerHTML = `
        
        <button type="button"
          data-action="click->image-upload#removeImage"
          data-filename="${file.name}"
          class="absolute top-0 right-0 bg-red-500 text-white rounded-full w-6 h-6">
          x
        </button>
      `
      this.previewTarget.appendChild(div)
    }

    reader.readAsDataURL(file)
  }

  removeImage(event) {
    const filename = event.currentTarget.dataset.filename
    this.selectedFiles = this.selectedFiles.filter(f => f.name !== filename)
    event.currentTarget.parentElement.remove()
  }

  async upload() {
    const formData = new FormData()

    this.selectedFiles.forEach((file, index) => {
      formData.append(`images[${index}]`, file)
    })

    try {
      const response = await fetch('/api/upload', {
        method: 'POST',
        body: formData,
        headers: {
          'X-CSRF-Token': this.csrfToken()
        }
      })

      const data = await response.json()
      return data.image_urls
    } catch (error) {
      console.error('Upload failed:', error)
    }
  }

  csrfToken() {
    return document.querySelector('[name="csrf-token"]').content
  }
}

```

5. Filter Controller (Mobile Bottom Sheet)

```
// app/javascript/controllers/filter_controller.js
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["modal", "form"]
  static values = { url: String }

  open() {
    this.modalTarget.classList.remove('translate-y-full')
    document.body.classList.add('overflow-hidden')
  }

  close() {
    this.modalTarget.classList.add('translate-y-full')
    document.body.classList.remove('overflow-hidden')
  }

  async apply(event) {
    event.preventDefault()

    const formData = new FormData(this.formTarget)
    const params = new URLSearchParams(formData)

    const url = `${this.urlValue}?${params.toString()}`

    // Use Turbo to navigate and update the page
    Turbo.visit(url)

    this.close()
  }

  reset() {
    this.formTarget.reset()
    this.apply({ preventDefault: () => {} })
  }
}
```

ViewComponent Architecture

Card Components

PostCardComponent:

```
# app/components/post_card_component.rb
class PostCardComponent < ViewComponent::Base
  def initialize(post:, current_user: nil)
    @post = post
    @current_user = current_user
  end

  def hearted?
    @current_user&.hearted?(@post)
  end

  def author_avatar
    @post.user.avatar_url || default_avatar
  end

  private

  def default_avatar
    "https://ui-avatars.com/api/?name=#{@post.user.username}"
  end
end
```

```

<!-- app/components/post_card_component.html.erb -->
<div class="post-card bg-white rounded-lg shadow-sm p-4 mb-4">
  <!-- Header -->
  <div class="flex items-center mb-3">
    
    <div class="flex-1">
      <div class="flex items-center">
        <span class="font-semibold text-gray-900"><%= @post.user.username %></span>
        <span class="text-gray-500 text-sm ml-2">• <%= time_ago_in_words(@post.created_at) %> ago</span>
      </div>
      <div class="text-sm text-blue-600">r/<%= @post.community.name %></div>
    </div>
  </div>

  <!-- Title -->
  <h2 class="text-lg font-semibold mb-2 text-gray-900">
    <%= link_to @post.title, post_path(@post), data: { turbo_frame: "_top" } %>
  </h2>

  <!-- Body (truncated) -->
  <% if @post.body.present? %>
    <p class="text-gray-700 mb-3 line-clamp-3"><%= @post.body %></p>
  <% end %>

  <!-- Images -->
  <% if @post.post_images.any? %>
    <div class="mb-3">
      <%= image_tag @post.post_images.first.image_url, class: "w-full rounded-lg" %>
    </div>
  <% end %>

  <!-- Actions -->
  <div class="flex items-center space-x-4 text-gray-600">
    <%= render HeartButtonComponent.new(heartable: @post, current_user: @current_user) %>

    <button class="flex items-center space-x-1 hover:text-blue-600">
      <span>🔍</span>
      <span><%= @post.comments_count %></span>
    </button>

    <button class="flex items-center space-x-1 hover:text-green-600">
      <span>🔍</span>
      <span>Share</span>
    </button>
  </div>
</div>

```

Hotwire Turbo Implementation

Turbo Frames for Modals

```

<!-- app/views/posts/index.html.erb -->
<div id="posts-container">
  <% @posts.each do |post| %>
    <%= turbo_frame_tag dom_id(post), src: post_path(post) do %>
      <%= render PostCardComponent.new(post: post, current_user: current_user) %>
    <% end %>
  <% end %>
</div>

<!-- Modal for post detail -->
<turbo-frame id="modal" class="fixed inset-0 bg-black bg-opacity-50 hidden z-50">
  <!-- Post detail loaded here -->
</turbo-frame>

```

Turbo Streams for Real-Time Updates

```

# app/controllers/posts_controller.rb
def create
  @post = current_user.posts.build(post_params)

  if @post.save
    # Broadcast to all community members' feeds
    broadcast_to_feeds(@post)

    respond_to do |format|
      format.turbo_stream
      format.html { redirect_to @post }
    end
  else
    render :new, status: :unprocessable_entity
  end
end

private

def broadcast_to_feeds(post)
  post.community.members.each do |member|
    broadcast_append_to(
      "user_feed_#{member.id}",
      target: "posts-container",
      partial: "posts/post_card",
      locals: { post: post }
    )
  end
end

```

Responsive Design Strategy

Mobile-First Approach

Breakpoints (Tailwind CSS)

```
// tailwind.config.js
module.exports = {
  theme: {
    screens: {
      'sm': '640px',    // Small phones in landscape, large phones in portrait
      'md': '768px',    // Tablets in portrait
      'lg': '1024px',   // Tablets in landscape, small laptops
      'xl': '1280px',   // Desktop
      '2xl': '1536px'  // Large desktop
    }
  }
}
```

Design System

Typography Scale:

```
/* Base: 16px (1rem) */
.text-xs    { font-size: 0.75rem; } /* 12px */
.text-sm    { font-size: 0.875rem; } /* 14px */
.text-base   { font-size: 1rem; }    /* 16px */
.text-lg     { font-size: 1.125rem; } /* 18px */
.text-xl     { font-size: 1.25rem; }  /* 20px */
.text-2xl    { font-size: 1.5rem; }   /* 24px */
.text-3xl    { font-size: 1.875rem; } /* 30px */
```

Spacing Scale:

```
.p-2  { padding: 0.5rem; } /* 8px */
.p-4  { padding: 1rem; }   /* 16px */
.p-6  { padding: 1.5rem; } /* 24px */
.p-8  { padding: 2rem; }   /* 32px */
```

Touch Targets:

- Minimum tap target size: **48x48px** (iOS/Android standard)
- Interactive elements: **44x44px minimum**
- Spacing between tappable elements: **8px minimum**

Navigation Patterns

Mobile Navigation (Bottom Tab Bar)

```

<!-- app/views/layouts/_mobile_nav.html.erb -->
<nav class="fixed bottom-0 left-0 right-0 bg-white border-t border-gray-200 lg:hidden z-50">
  <div class="flex justify-around items-center h-16">
    <%= link_to root_path, class: "flex flex-col items-center justify-center flex-1 text-gray-600 #{current_page?(root_path) ? 'text-white' : ''}">
      <span class="text-2xl">🏠</span>
      <span class="text-xs mt-1">Home</span>
    <% end %>

    <%= link_to kpop_groups_path, class: "flex flex-col items-center justify-center flex-1 text-gray-600 #{current_page?(kpop_groups_path) ? 'text-white' : ''}">
      <span class="text-2xl">🎵</span>
      <span class="text-xs mt-1">Discover</span>
    <% end %>

    <button class="flex flex-col items-center justify-center flex-1 text-gray-600"
      data-controller="modal"
      data-action="click->modal#open">
      <span class="text-2xl">📧</span>
      <span class="text-xs mt-1">Post</span>
    </button>

    <%= link_to notifications_path, class: "flex flex-col items-center justify-center flex-1 text-gray-600 relative" do %>
      <span class="text-2xl">🔔</span>
      <% if current_user.unread_notifications_count > 0 %>
        <span class="absolute top-2 right-6 bg-red-500 text-white text-xs rounded-full w-5 h-5 flex items-center justify-center">
          <%= current_user.unread_notifications_count %>
        </span>
      <% end %>
      <span class="text-xs mt-1">Notifications</span>
    <% end %>

    <%= link_to user_path(current_user), class: "flex flex-col items-center justify-center flex-1 text-gray-600" do %>
      
      <span class="text-xs mt-1">Profile</span>
    <% end %>
  </div>
</nav>

```

Desktop Navigation (Sidebar)

```
<!-- app/views/layouts/_desktop_nav.html.erb -->
<aside class="hidden lg:block fixed left-0 top-0 h-full w-64 bg-white border-r border-gray-200 overflow-y-auto">
  <div class="p-4">
    <h1 class="text-2xl font-bold text-blue-600 mb-6">K-Pop Universe</h1>

    <!-- Quick Links -->
    <nav class="space-y-2 mb-6">
      <%= link_to "Home", root_path, class: "block px-4 py-2 rounded hover:bg-gray-100" %>
      <%= link_to "Popular", popular_path, class: "block px-4 py-2 rounded hover:bg-gray-100" %>
      <%= link_to "Discover Groups", kpop_groups_path, class: "block px-4 py-2 rounded hover:bg-gray-100" %>
    </nav>

    <!-- Followed Communities -->
    <div class="mb-6">
      <h3 class="text-sm font-semibold text-gray-500 uppercase mb-2">Your Communities</h3>
      <%= current_user.communities.each do |community| %>
        <%= link_to community_path(communit), class: "flex items-center px-4 py-2 rounded hover:bg-gray-100" do %>
          
          <span class="text-sm">r/<%= community.name %></span>
        <% end %>
      <% end %>
    </div>
  </div>
</aside>
```

Responsive Layout Examples

Post Feed Layout

```
<!-- Mobile: Single column, Desktop: Feed + Sidebar -->
<div class="lg:ml-64 min-h-screen bg-gray-50">
  <div class="max-w-7xl mx-auto px-4 py-6 lg:px-8">
    <div class="lg:grid lg:grid-cols-3 lg:gap-6">
      <!-- Main Feed (mobile: full width, desktop: 2/3) -->
      <div class="lg:col-span-2">
        <%= render "posts/feed" %>
      </div>

      <!-- Sidebar (mobile: hidden, desktop: 1/3) -->
      <div class="hidden lg:block">
        <%= render "sidebar/trending_communities" %>
        <%= render "sidebar/trending_groups" %>
      </div>
    </div>
  </div>
</div>

<!-- Mobile Bottom Navigation -->
<%= render "layouts/mobile_nav" %>
```

Performance Optimizations for Mobile

Image Optimization

1. Responsive Images:

```
<%= image_tag post.image_url,
      srcset: "#{post.image_url_small} 320w,
              #{post.image_url_medium} 768w,
              #{post.image_url_large} 1024w",
      sizes: "(max-width: 640px) 100vw,
              (max-width: 1024px) 50vw,
              33vw",
      class: "w-full h-auto",
      loading: "lazy" %>
```

2. WebP Format

```
# Use AWS Lambda or ImageMagick to convert uploads to WebP
# Serve WebP to supported browsers, fallback to JPEG/PNG
```

3. CDN Delivery:

- Store all images on S3
- Serve via CloudFront CDN
- Enable Brotli/Gzip compression

CSS Optimization

```
# Purge unused Tailwind CSS classes in production
npm run build:css -- --minify
```

JavaScript Bundle Size

- **Stimulus.js**: ~40KB (gzipped)
- **Hotwire Turbo**: ~25KB (gzipped)
- **Total JS**: <100KB (target)

Lazy Loading

```
// Lazy load images, iframes, and below-the-fold content

```

Cloud Infrastructure

AWS Architecture

Core Services

1. EC2 (Web Server):

- **Instance Type**: t3.medium (2 vCPU, 4GB RAM) initially, scale to t3.large/c5.large as needed
- **OS**: Ubuntu 22.04 LTS
- **Auto Scaling**: Target 70% CPU utilization
- **Load Balancer**: Application Load Balancer (ALB) for HTTPS termination

2. RDS (PostgreSQL Database):

- **Instance**: db.t4g.medium (2 vCPU, 4GB RAM) initially
- **Engine**: PostgreSQL 15.x
- **Multi-AZ**: Enabled for high availability
- **Backup**: Daily automated backups, 7-day retention
- **Read Replicas**: Add as traffic grows

3. S3 (Object Storage):

- **Buckets**:
 - kpopuniverse-uploads - User-uploaded images/videos
 - kpopuniverse-assets - Static assets (CSS, JS, fonts)
 - kpopuniverse-backups - Database backups
- **Lifecycle Policies**: Transition old images to Glacier after 1 year

4. CloudFront (CDN):
- **Purpose:** Serve static assets and images with low latency globally
 - **Origin:** S3 buckets
 - **Cache Behavior:**
 - Static assets: 1 year TTL
 - User uploads: 1 day TTL
 - **Compression:** Enable Gzip and Brotli
5. ElastiCache (Redis):
- **Instance:** cache.t4g.micro initially
 - **Purpose:**
 - Session storage
 - ActionCable adapter for real-time features
 - Fragment caching
 - Job queue (Sidekiq)
6. SES (Email Service):
- **Purpose:** Transactional emails (password reset, notifications)
 - **Configuration:** Domain verification, DKIM, SPF records
7. CloudWatch (Monitoring):
- **Metrics:** CPU, memory, disk, network, application errors
 - **Alarms:** High CPU (>80%), high error rate (>5%)
 - **Logs:** Application logs, access logs, error logs

Architecture Diagram



Deployment Strategy

1. Initial Setup (Terraform)

```
# terraform/main.tf
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbfaffe1f0" # Ubuntu 22.04
  instance_type = "t3.medium"

  tags = {
    Name = "kpopuniverse-web"
  }
}

resource "aws_db_instance" "postgres" {
  allocated_storage = 100
  engine            = "postgres"
  engine_version    = "15.3"
  instance_class    = "db.t4g.medium"
  db_name           = "kpopuniverse_production"
  username          = var.db_username
  password          = var.db_password
  multi_az          = true
  backup_retention_period = 7

  tags = {
    Name = "kpopuniverse-db"
  }
}

resource "aws_s3_bucket" "uploads" {
  bucket = "kpopuniverse-uploads"

  tags = {
    Name = "kpopuniverse-uploads"
  }
}
```

2. Continuous Deployment (GitHub Actions)

```
# .github/workflows/deploy.yml
name: Deploy to AWS

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Ruby
        uses: ruby/setup-ruby@v1
        with:
          ruby-version: 3.2

      - name: Install dependencies
        run: bundle install

      - name: Run tests
        run: bundle exec rspec

      - name: Build assets
        run: bundle exec rails assets:precompile

      - name: Deploy to EC2
        uses: appleboy/ssh-action@master
        with:
          host: ${ secrets.EC2_HOST }
          username: ubuntu
          key: ${ secrets.SSH_PRIVATE_KEY }
          script: |
            cd /var/www/kpopuniverse
            git pull origin main
            bundle install
            bundle exec rails db:migrate
            bundle exec rails assets:precompile
            sudo systemctl restart puma
            sudo systemctl restart sidekiq
```

Environment Configuration

```
# .env.production
RAILS_ENV=production
RAILS_SERVE_STATIC_FILES=false
RAILS_LOG_TO_STDOUT=true

# Database
DATABASE_URL=postgresql://user:pass@kpopuniverse.abc123.us-east-1.rds.amazonaws.com/kpopuniverse_production

# Redis
REDIS_URL=redis://kpopuniverse.abc123.use1.cache.amazonaws.com:6379/0

# AWS
AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
S3_BUCKET_NAME=kpopuniverse-uploads

# CloudFront
CLOUDFRONT_DISTRIBUTION_ID=E1234567890ABC

# OAuth
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
APPLE_CLIENT_ID=...
APPLE_TEAM_ID=...
FACEBOOK_APP_ID=...
FACEBOOK_APP_SECRET=...

# Domain
APP_DOMAIN=kpopuniverse.com
```

Monitoring & Alerting

CloudWatch Alarms

1. **High CPU Usage:** >80% for 5 minutes → Alert
2. **High Error Rate:** >5% of requests → Alert
3. **Database Connections:** >80% of max connections → Alert
4. **Disk Space:** <10% free space → Alert
5. **Memory Usage:** >90% for 5 minutes → Alert

Application Monitoring (Datadog or New Relic)

- **APM:** Track slow requests, database queries
- **Error Tracking:** Sentry integration for exception monitoring
- **Custom Metrics:** Feed build time, notification delivery rate

Key Features Breakdown

Feature 1: Personalized Feed System

Requirements

- Pre-built feeds for instant loading (<1s)
- Push-based updates (not pull-based)
- Real-time new post notifications
- Priority-based positioning (not just chronological)
- Sync engagement metrics at thresholds

Implementation

1. **Feed Item Model** (see Data Models section)
2. **Feed Builder Service:**

```

# app/services/feed_builder_service.rb
class FeedBuilderService
  def initialize(user)
    @user = user
  end

  # Build initial feed when user follows a community
  def build_feed_for_community(community)
    recent_posts = community.posts
                        .where('created_at > ?', 7.days.ago)
                        .order(created_at: :desc)
                        .limit(50)

    recent_posts.each do |post|
      add_post_to_feed(post)
    end
  end

  # Add new post to user's feed
  def add_post_to_feed(post)
    FeedItem.create!(
      user: @user,
      post: post,
      post_title: post.title,
      post_type: post.post_type,
      community_id: post.community_id,
      community_name: post.community.name,
      author_id: post.user_id,
      author_username: post.user.username,
      cached_hearts_count: post.hearts_count,
      cached_comments_count: post.comments_count,
      priority_score: calculate_priority_score(post)
    )
  end

  # Priority score: engagement + recency
  def calculate_priority_score(post)
    hours_since_creation = (Time.current - post.created_at) / 1.hour

    engagement_score = Math.log10(post.hearts_count + 1) +
                        Math.log10(post.comments_count + 1)

    recency_penalty = hours_since_creation / 24.0

    engagement_score - recency_penalty
  end
end

```

3. Feed Update Job (Background job to push posts to feeds):

```

# app/jobs/push_post_to_feeds_job.rb
class PushPostToFeedsJob < ApplicationJob
  queue_as :high_priority

  def perform(post_id)
    post = Post.find(post_id)
    community = post.community

    # Get all community members
    members = community.members.where.not(id: post.user_id) # Exclude author

    # Batch insert feed items for performance
    feed_items = members.map do |member|
      {
        user_id: member.id,
        post_id: post.id,
        post_title: post.title,
        post_type: post.post_type,
        community_id: community.id,
        community_name: community.name,
        author_id: post.user_id,
        author_username: post.user.username,
        cached_hearts_count: 0,
        cached_comments_count: 0,
        priority_score: FeedBuilderService.new(member).calculate_priority_score(post),
        created_at: Time.current
      }
    end

    FeedItem.insert_all(feed_items) if feed_items.any?

    # Broadcast to ActionCable for real-time updates
    members.each do |member|
      broadcast_new_post_to_user(member, post)
    end
  end

  private

  def broadcast_new_post_to_user(user, post)
    ActionCable.server.broadcast(
      "user_feed_#{user.id}",
      {
        type: 'new_post',
        html: ApplicationController.render(
          partial: 'posts/post_card',
          locals: { post: post }
        )
      }
    )
  end
end

```

4. Engagement Sync Job (Update cached metrics at thresholds):

```

# app/jobs/sync_engagement_job.rb
class SyncEngagementJob < ApplicationJob
  queue_as :low_priority

  SYNC_THRESHOLDS = [10, 50, 100, 500, 1000, 5000].freeze

  def perform(post_id)
    post = Post.find(post_id)
    current_hearts = post.hearts_count

    # Check if we've crossed a threshold
    return unless should_sync?(post, current_hearts)

    # Update all feed items for this post
    FeedItem.where(post_id: post_id).update_all(
      cached_hearts_count: post.hearts_count,
      cached_comments_count: post.comments_count
    )
  end

  private

  def should_sync?(post, current_hearts)
    SYNC_THRESHOLDS.any? do |threshold|
      post.hearts_count >= threshold &&
      post.previous_changes[:hearts_count]&.first.to_i < threshold
    end
  end
end

```

5. Feed Controller:

```

# app/controllers/feed_controller.rb
class FeedController < ApplicationController
  def home
    @feed_items = current_user.feed_items
                                .includes(post: [:user, :community])
                                .order(priority_score: :desc)
                                .page(params[:page])
                                .per(25)

    respond_to do |format|
      format.html
      format.turbo_stream
    end
  end
end

```

Feature 2: K-Pop Group Discovery

Requirements

- Filterable by company and group type
- Detailed profiles (members, photos, discography, social links)
- Follow/unfollow groups
- Search functionality

Implementation

1. Discovery Page:

```

<!-- app/views/kpop_groups/index.html.erb -->
<div class="max-w-7xl mx-auto px-4 py-6">
  <h1 class="text-3xl font-bold mb-6">Get to Know K-Pop Groups</h1>

  <!-- Filters -->
  <div class="bg-white rounded-lg shadow-sm p-4 mb-6">
    <%= form_with url: kpop_groups_path, method: :get,
      data: { controller: "filter" } do |f| %>

    <div class="grid grid-cols-1 md:grid-cols-3 gap-4">
      <!-- Company Filter -->
      <div>
        <%= f.label :company_id, "Company", class: "block text-sm font-medium mb-2" %>
        <%= f.select :company_id,
          options_from_collection_for_select(@companies, :id, :name, params[:company_id]),
          { include_blank: "All Companies" },
          class: "w-full border border-gray-300 rounded-lg px-4 py-2" %>
      </div>

      <!-- Group Type Filter -->
      <div>
        <%= f.label :group_type, "Group Type", class: "block text-sm font-medium mb-2" %>
        <%= f.select :group_type,
          options_for_select(KpopGroup.group_types.keys.map { |k| [k.humanize, k] }, params[:group_type]),
          { include_blank: "All Types" },
          class: "w-full border border-gray-300 rounded-lg px-4 py-2" %>
      </div>

      <!-- Status Filter -->
      <div>
        <%= f.label :status, "Status", class: "block text-sm font-medium mb-2" %>
        <%= f.select :status,
          options_for_select(KpopGroup.statuses.keys.map { |k| [k.humanize, k] }, params[:status]),
          { include_blank: "All Statuses" },
          class: "w-full border border-gray-300 rounded-lg px-4 py-2" %>
      </div>
    </div>

    <div class="mt-4">
      <%= f.submit "Apply Filters", class: "bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700" %>
      <%= link_to "Reset", kpop_groups_path, class: "ml-4 text-gray-600 hover:text-gray-900" %>
    </div>
  <% end %>
</div>

  <!-- Groups Grid -->
  <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-6">
    <% @kpop_groups.each do |group| %>
      <%= render GroupCardComponent.new(group: group, current_user: current_user) %>
    <% end %>
  </div>

  <!-- Pagination -->
  <%= paginate @kpop_groups %>
</div>

```

2. Group Detail Page:

```

<!-- app/views/kpop_groups/show.html.erb -->
<div class="max-w-5xl mx-auto px-4 py-6">
  <!-- Banner -->
  <div class="relative h-64 bg-cover bg-center rounded-lg mb-6"
    style="background-image: url('<%= @group.banner_image_url %>')">
    <div class="absolute inset-0 bg-black bg-opacity-40 rounded-lg"></div>
  </div>

```

```

<!-- Profile -->
<div class="bg-white rounded-lg shadow-sm p-6 -mt-32 relative z-10">
  <div class="flex items-start">
    

    <div class="ml-6 flex-1">
      <h1 class="text-3xl font-bold"><%= @group.name %></h1>
      <p class="text-gray-600 text-lg mb-2"><%= @group.korean_name %></p>

      <div class="flex items-center space-x-4 text-sm text-gray-600 mb-4">
        <span><%= @group.group_type.humanize %></span>
        <span>•</span>
        <span>Debut: <%= @group.debut_date.strftime('%B %d, %Y') %></span>
        <span>•</span>
        <span><%= @group.company.name %></span>
      </div>

      <%= render FollowButtonComponent.new(group: @group, current_user: current_user) %>
    </div>
  </div>

  <!-- Fandom Info -->
  <% if @group.fandom_name.present? %>
    <div class="mt-4 p-4 bg-gray-50 rounded-lg">
      <span class="font-semibold">Fandom:</span> <%= @group.fandom_name %>
      <% if @group.official_colors.present? %>
        <span class="ml-4">
          <span class="font-semibold">Official Colors:</span>
          <% @group.official_colors.split(',').each do |color| %>
            <span class="inline-block w-6 h-6 rounded-full ml-2 border border-gray-300"
              style="background-color: <%= color.strip %>"></span>
          <% end %>
        </span>
      <% end %>
    </div>
  <% end %>

  <!-- Description -->
  <div class="mt-6">
    <h2 class="text-xl font-semibold mb-3">About</h2>
    <p class="text-gray-700 leading-relaxed"><%= @group.description %></p>
  </div>

  <!-- Members -->
  <div class="mt-6">
    <h2 class="text-xl font-semibold mb-4">Members</h2>
    <div class="grid grid-cols-2 md:grid-cols-4 gap-4">
      <% @group.group_members.each do |member| %>
        <div class="text-center">
          
          <h3 class="font-semibold"><%= member.stage_name %></h3>
          <p class="text-sm text-gray-600"><%= member.position %></p>
        </div>
      <% end %>
    </div>
  </div>

  <!-- Social Links -->
  <div class="mt-6">
    <h2 class="text-xl font-semibold mb-3">Follow <%= @group.name %></h2>
    <div class="flex space-x-4">
      <% if @group.youtube_url.present? %>

```

```

      <%= link_to @group.youtube_url, target: "_blank",
        class: "text-red-600 hover:text-red-700 text-2xl" do %>
         YouTube
      <% end %>
    <% end %>

    <% if @group.twitter_url.present? %>
      <%= link_to @group.twitter_url, target: "_blank",
        class: "text-blue-500 hover:text-blue-600 text-2xl" do %>
         Twitter
      <% end %>
    <% end %>

    <% if @group.instagram_url.present? %>
      <%= link_to @group.instagram_url, target: "_blank",
        class: "text-pink-600 hover:text-pink-700 text-2xl" do %>
         Instagram
      <% end %>
    <% end %>

    <% if @group.spotify_url.present? %>
      <%= link_to @group.spotify_url, target: "_blank",
        class: "text-green-600 hover:text-green-700 text-2xl" do %>
         Spotify
      <% end %>
    <% end %>
  </div>
</div>
</div>

<!-- Related Community -->
<% if @related_community %>
  <div class="mt-6 bg-blue-50 border border-blue-200 rounded-lg p-4">
    <h3 class="font-semibold mb-2">Join the <%= @group.name %> Community</h3>
    <p class="text-sm text-gray-700 mb-3">
      Discuss <%= @group.name %> with other fans in r/<%= @related_community.name %>
    </p>
    <%= link_to "Visit Community →", community_path(@related_community),
      class: "inline-block bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700" %>
  </div>
<% end %>
</div>

```

Feature 3: Real-Time Notifications

Requirements

- Instant notifications for comment replies
- Batched notifications for hearts (e.g., "Your post got 10 hearts!")
- Milestone notifications (e.g., "Your post reached 100 hearts!")
- In-app notification center + web push notifications

Implementation

1. Notification Service:

```

# app/services/notification_service.rb
class NotificationService
  def self.notify_comment_reply(comment)
    parent_author = comment.parent.user
    return if parent_author == comment.user # Don't notify self

    notification = Notification.create!(
      user: parent_author,
      notification_type: 'comment_reply',
      subject: comment,
      actor: comment.user,
      message: "#{comment.user.username} replied to your comment"
    )

    # Broadcast to ActionCable
    NotificationChannel.broadcast_to(
      parent_author,
      {
        type: 'new_notification',
        html: ApplicationController.render(
          partial: 'notifications/notification',
          locals: { notification: notification }
        )
      }
    )

    # Send web push notification
    WebPushService.send_notification(parent_author, notification)
  end

  def self.notify_milestone(post, milestone_count)
    notification = Notification.create!(
      user: post.user,
      notification_type: 'milestone',
      subject: post,
      message: "Your post reached #{milestone_count} hearts!"
    )

    NotificationChannel.broadcast_to(post.user, {
      type: 'new_notification',
      html: ApplicationController.render(
        partial: 'notifications/notification',
        locals: { notification: notification }
      )
    })
  end
end

```

2. ActionCable Channel:

```

# app/channels/notification_channel.rb
class NotificationChannel < ApplicationCable::Channel
  def subscribed
    stream_for current_user
  end

  def unsubscribed
    stop_all_streams
  end
end

```

3. Stimulus Controller for Notifications:

```

// app/javascript/controllers/notification_controller.js
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = ["badge", "list"]
  static values = { userId: Number }

  connect() {
    this.subscription = this.createSubscription()
    this.requestPermission()
  }

  disconnect() {
    this.subscription?.unsubscribe()
  }

  createSubscription() {
    return App.cable.subscriptions.create(
      { channel: "NotificationChannel", user_id: this.userIdValue },
      {
        received: (data) => {
          if (data.type === 'new_notification') {
            this.addNotification(data.html)
            this.incrementBadge()
            this.showToast(data.message)
          }
        }
      }
    )
  }

  addNotification(html) {
    this.listTarget.insertAdjacentHTML('afterbegin', html)
  }

  incrementBadge() {
    const count = parseInt(this.badgeTarget.textContent) || 0
    this.badgeTarget.textContent = count + 1
    this.badgeTarget.classList.remove('hidden')
  }

  showToast(message) {
    // Show brief notification toast
    const toast = document.createElement('div')
    toast.className = 'fixed top-4 right-4 bg-blue-600 text-white px-6 py-3 rounded-lg shadow-lg z-50'
    toast.textContent = message
    document.body.appendChild(toast)

    setTimeout(() => toast.remove(), 3000)
  }

  async requestPermission() {
    if ('Notification' in window && Notification.permission === 'default') {
      await Notification.requestPermission()
    }
  }
}

```

Feature 4: Community Creation & Moderation

Requirements

- Any registered user can create communities instantly

- Customizable community appearance (icon, banner, rules)
- Moderator appointment by community creator
- Moderation tools (remove posts, pin posts, ban users)

Implementation

1. Community Creation Flow:

```

<!-- app/views/communities/new.html.erb -->
<div class="max-w-2xl mx-auto px-4 py-6">
  <h1 class="text-3xl font-bold mb-6">Create a Community</h1>

  <%= form_with model: @community, class: "space-y-6" do |f| %>
    <!-- Community Name -->
    <div>
      <%= f.label :name, "Community Name", class: "block text-sm font-medium mb-2" %>
      <div class="flex items-center">
        <span class="text-gray-500 mr-2">r/</span>
        <%= f.text_field :name,
          placeholder: "bts",
          class: "flex-1 border border-gray-300 rounded-lg px-4 py-2",
          data: { controller: "availability-check" } %>
      </div>
      <p class="text-xs text-gray-600 mt-1">
        3-21 characters, lowercase letters, numbers, and underscores only
      </p>
    </div>

    <!-- Display Name -->
    <div>
      <%= f.label :display_name, "Display Name", class: "block text-sm font-medium mb-2" %>
      <%= f.text_field :display_name,
        placeholder: "BTS - Bangtan Sonyeondan",
        class: "w-full border border-gray-300 rounded-lg px-4 py-2" %>
    </div>

    <!-- Description -->
    <div>
      <%= f.label :description, "Description", class: "block text-sm font-medium mb-2" %>
      <%= f.text_area :description,
        rows: 4,
        placeholder: "A community for discussing BTS music, news, and more!",
        class: "w-full border border-gray-300 rounded-lg px-4 py-2" %>
    </div>

    <!-- Community Icon -->
    <div>
      <%= f.label :icon, "Community Icon", class: "block text-sm font-medium mb-2" %>
      <%= f.file_field :icon,
        accept: "image/*",
        data: { controller: "image-upload" },
        class: "w-full" %>
    </div>

    <!-- Visibility -->
    <div>
      <%= f.label :visibility, "Visibility", class: "block text-sm font-medium mb-2" %>
      <%= f.select :visibility,
        options_for_select(['Public', 'public_community'], ['Private', 'private_community']),
        {},
        class: "w-full border border-gray-300 rounded-lg px-4 py-2" %>
    </div>

    <div class="flex justify-end space-x-4">
      <%= link_to "Cancel", communities_path, class: "px-6 py-2 border border-gray-300 rounded-lg hover:bg-gray-50" %>
      <%= f.submit "Create Community", class: "px-6 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700" %>
    </div>
  <% end %>
</div>

```

2. Moderation Dashboard:

```

<!-- app/views/communities/moderate.html.erb -->
<div class="max-w-7xl mx-auto px-4 py-6">
  <h1 class="text-3xl font-bold mb-6">Moderate r/<%= @community.name %></h1>

  <!-- Moderation Queue -->
  <div class="bg-white rounded-lg shadow-sm p-6 mb-6">
    <h2 class="text-xl font-semibold mb-4">Reported Content</h2>

    <% @reported_posts.each do |post| %>
      <div class="border-b border-gray-200 py-4">
        <div class="flex justify-between items-start">
          <div class="flex-1">
            <h3 class="font-semibold"><%= post.title %></h3>
            <p class="text-sm text-gray-600">by u/<%= post.user.username %></p>
            <p class="text-sm text-red-600 mt-1">
              <%= post.reports.count %> reports: <%= post.reports.pluck(:reason).join(', ') %>
            </p>
          </div>

          <div class="flex space-x-2">
            <%= button_to "Approve", approve_post_path(post), method: :patch,
              class: "px-4 py-2 bg-green-600 text-white rounded hover:bg-green-700" %>
            <%= button_to "Remove", remove_post_path(post), method: :patch,
              class: "px-4 py-2 bg-red-600 text-white rounded hover:bg-red-700" %>
          </div>
        </div>
      </div>
    <% end %>
  </div>

  <!-- Moderator Management -->
  <div class="bg-white rounded-lg shadow-sm p-6">
    <h2 class="text-xl font-semibold mb-4">Moderators</h2>

    <ul class="space-y-2">
      <% @community.moderators.each do |mod| %>
        <li class="flex items-center justify-between">
          <span>u/<%= mod.username %></span>
          <% if current_user == @community.creator %>
            <%= button_to "Remove", remove_moderator_path(@community, mod),
              method: :delete,
              class: "text-red-600 hover:text-red-700 text-sm" %>
          <% end %>
        </li>
      <% end %>
    </ul>

    <% if current_user == @community.creator %>
      <%= form_with url: add_moderator_community_path(@community), method: :post, class: "mt-4" do |f| %>
        <%= f.text_field :username, placeholder: "Username", class: "border border-gray-300 rounded-lg px-4 py-2" %>
        <%= f.submit "Add Moderator", class: "ml-2 px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700" %>
      <% end %>
    <% end %>
  </div>
</div>

```

Implementation Phases

Phase 1: MVP (Months 1-3)

Goal: Launch a functional web app with core community features

Week 1-2: Project Setup & Infrastructure

- ☐ Rails 7.x app initialization
- ☐ PostgreSQL database setup
- ☐ Devise authentication with email/password
- ☐ Basic AWS infrastructure (EC2, RDS, S3)
- ☐ Tailwind CSS configuration
- ☐ Stimulus.js setup

Week 3-4: User Authentication & Profiles

- ☐ User registration/login
- ☐ User profile pages
- ☐ Avatar upload to S3
- ☐ Guest browsing capability
- ☐ Mobile-responsive auth forms

Week 5-6: Communities & Posts

- ☐ Community model & CRUD
- ☐ Post model & CRUD (text posts only)
- ☐ Community membership (join/leave)
- ☐ Post listing in communities
- ☐ Mobile-responsive layouts

Week 7-8: Engagement Features

- ☐ Heart system (posts & comments)
- ☐ Comment system (nested replies)
- ☐ Karma/points system
- ☐ Share functionality
- ☐ Save/bookmark posts

Week 9-10: Feed System

- ☐ Feed item model & denormalization
- ☐ Feed builder service
- ☐ Personalized home feed
- ☐ Feed pagination/infinite scroll
- ☐ Real-time feed updates (Turbo Streams)

Week 11-12: Polish & Launch Prep

- ☐ Mobile UI refinements
- ☐ Performance optimization
- ☐ Error handling & validations
- ☐ Basic moderation tools
- ☐ Deploy to AWS production
- ☐ Beta testing with 50-100 users

Deliverable: Working web app with communities, posts, comments, hearts, and personalized feeds

Phase 2: Enhancements (Months 4-6)

Goal: Add rich media, discovery, and advanced features

Month 4: Media & Content Types

- ☐ Image uploads (multi-image posts)
- ☐ Video uploads + YouTube/TikTok embeds
- ☐ Link posts with preview cards
- ☐ Poll posts
- ☐ CloudFront CDN integration
- ☐ Image optimization pipeline

Month 5: K-Pop Discovery

- ☐ K-Pop group model & database

- ☐ Company model
- ☐ Group member profiles
- ☐ Discovery page with filters
- ☐ Group detail pages
- ☐ Follow/unfollow groups
- ☐ Link groups to communities

Month 6: Notifications & Real-Time

- ☐ Notification model & system
- ☐ ActionCable for real-time updates
- ☐ Web push notifications
- ☐ Email notifications (SES)
- ☐ Notification preferences
- ☐ Batched & milestone notifications

Deliverable: Feature-complete web app with media, discovery, and notifications

Phase 3: Growth & Scaling (Months 7-9)

Goal: Scale infrastructure and add growth features

Month 7: Search & SEO

- ☐ Full-text search (PostgreSQL FTS or Elasticsearch)
- ☐ Search posts, communities, users, groups
- ☐ SEO optimization (meta tags, sitemaps, structured data)
- ☐ Open Graph tags for social sharing

Month 8: Advanced Moderation

- ☐ Moderation dashboard
- ☐ Content reporting system
- ☐ AutoMod rules (spam detection)
- ☐ User banning/warnings
- ☐ Community analytics

Month 9: Performance & Reliability

- ☐ Database query optimization
- ☐ Fragment caching with Redis
- ☐ Auto-scaling EC2 instances
- ☐ CloudWatch monitoring & alerts
- ☐ Error tracking (Sentry)
- ☐ Load testing

Deliverable: Scalable platform ready for 100K+ users

Phase 4: Mobile Native (Months 10-12) - Optional

Goal: Launch native iOS/Android apps with Hotwire Native

Month 10: Hotwire Native Setup

- ☐ Hotwire Native iOS app
- ☐ Hotwire Native Android app
- ☐ Path configurations for native screens
- ☐ Deep linking setup

Month 11: Native Features

- ☐ Native navigation (tab bar, modals)
- ☐ Native camera integration
- ☐ Native push notifications
- ☐ Offline support (basic)
- ☐ App Store/Play Store setup

Month 12: Launch & Iterate

- ☐ Beta testing (TestFlight, Google Play Beta)
- ☐ App Store submissions
- ☐ User feedback iteration
- ☐ Analytics integration

Deliverable: Native iOS/Android apps in app stores

Appendix

Design Mockups (To Be Created)

- Homepage (guest vs logged-in)
- Community page
- Post detail page
- K-Pop group discovery
- Mobile navigation
- Notification center
- Community creation wizard

API Documentation (To Be Generated)

- Use `rswag gem` for Swagger/OpenAPI docs
- Document all endpoints for future mobile API usage

Testing Strategy

- **Unit Tests:** RSpec for models, services
- **Integration Tests:** RSpec for controllers, request specs
- **System Tests:** Capybara for end-to-end flows
- **Mobile Testing:** BrowserStack for cross-device testing
- **Load Testing:** k6 or Apache JMeter

Analytics & Metrics

- **Google Analytics 4:** User behavior tracking
- **Mixpanel:** Event tracking (post creation, hearts, shares)
- **Hotjar:** Heatmaps & session recordings (mobile UX)
- **Custom Dashboard:** Admin panel with key metrics

Conclusion

This PRD provides a comprehensive blueprint for building K-Pop Universe as a mobile-first, Reddit-style community platform. The phased approach ensures we deliver value quickly while maintaining quality and scalability.

Key Success Factors:

1. **Mobile-First Design:** Excellent mobile UX is non-negotiable
2. **Performance:** Pre-built feeds ensure instant load times
3. **Engagement:** Instagram-style hearts + Reddit-style communities
4. **Discovery:** Comprehensive K-Pop group database
5. **Real-Time:** ActionCable for live updates without page reloads

Next Steps:

1. Review and approve this PRD
2. Create detailed design mockups (Figma)
3. Set up project infrastructure (Rails app, AWS, GitHub)
4. Begin Phase 1 development (MVP in 3 months)

Let's build the ultimate K-Pop community platform together! 🚀🎵